```
$ java -jar build/libs/catalog-service-0.0.1-SNAPSHOT.jar
```

NOTE Another practical Gradle task is `build`, which combines the operations of the `bootJar` and `test` tasks.

Since the project contains the `spring-boot-starter-web` dependency, Spring Boot automatically configures an embedded Tomcat server. By looking at the logs in figure 3.4, you can see that one of the first execution steps is initializing a Tomcat server instance embedded in the application itself.

```
...Starting CatalogServiceApplication using Java 17
...No active profile set, falling back to 1 default profile: "default"
...Tomcat initialized with port(s): 9001 (http)
...Starting service [Tomcat]
...Starting Servlet engine: [Apache Tomcat/9.0.65]
...Initializing Spring embedded WebApplicationContext
...Root WebApplicationContext: initialization completed in 276 ms
...Tomcat started on port(s): 9001 (http) with context path ''
...Started CatalogServiceApplication in 0.529 seconds (JVM running for 0.735)
```

Figure 3.4 The startup logs from the Catalog Service application

In the next section, you'll learn more about how the embedded server works in Spring Boot. Before moving on, though, you can stop the application with Ctrl-C.

3.2.2 Understanding the thread-per-request model